Midterm Exam review

William Hendrix

Lecture 6

Midterm Exam review

Potential topics:

- Big-Oh formal definitions
  - ~~Properties~~
- Algorithm analysis
  - Iterative analysis
  - Recursion tree analysis
  - Master Theorem
- Data structures
  - Stacks and queues
  - Sets
  - Maps
  - Priority Queues/Heaps
  - Union-Find
  - Algorithm design
- Sorting and selection algorithms

Back

Summary: complexity and Big-Oh

- RAM model of computation
- Useful approximation of real-world behavior:
  - Basic instructions take same amount of time
  - Memory access is instantaneous
- Key aspect of complexity: asymptotic growth
  - How fast does the function grow?
  - Constant, logarithmic, linear, etc.?
- Big-Oh: classify functions according to growth rate

$f(n) = O(g(n))$ if and only if there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

$f(n) = \Omega(g(n))$ if and only if there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$.

$f(n) = \Theta(g(n))$ if and only if there exist positive constants c_1, c_2, and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

- *Analogy:* O, Ω, and Θ "act like" $\leq$, $\geq$, and $=$

Back

Big-Oh properties

- **Interrelationships:** O and Ω are "*opposite*", Θ is "*composite*"

  $$f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$$
  $$f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)) \Leftrightarrow f(n) = \Theta(g(n))$$
- **Reflexive**

  $$f(n) = O(f(n)), \text{ for any function } f$$
- *Θ only*: **Symmetric**

  $$f(n) = \Theta(g(n)) \rightarrow g(n) = \Theta(f(n))$$
- **Transitive**

  $$f(n) = O(g(n)) \text{ and } g(n) = O(h(n)) \rightarrow f(n) = O(h(n))$$
- **Ignore constant coefficients**

  $$\forall x > 0, xf(n) = O(f(n))$$
- **Ignore small terms**

  $$f(n) = O(g(n)) \rightarrow \Theta(f(n) + g(n)) = \Theta(g(n))$$
- **Envelopment** (+ and *)

  $$O(f(n)) + O(g(n)) = O(f(n) + g(n))$$
  $$O(f(n))O(g(n)) = O(f(n)g(n))$$

Back

Big-Oh exercise

- Prove the envelopment property of addition of Big-Oh:
  If $f_1(n) = O(g_1(n))$ and $f_2(n) = O(g_2(n))$,
  then $f_1(n) + f_2(n) = O(g_1(n) + g_2(n))$.

Back

Summary: algorithm analysis

- Identify loops and function calls
  - Everything else is *Θ(1)*
- *For loops*:
  - Estimate number of iterations
    - Incrementing by *c*: divide range by *c* to get iterations
    - Multiplying by *c:* take $\log_c$ of the end/start ratio
  - Estimate loop body running time
    - Might depend on iteration #
  - If iterations don't depend on i: # iterations * time per iteration
  - Otherwise: sum up all iterations

$$\sum_{i=1}^{x} i = \Theta(x^2) \qquad \sum_{i=1}^{x} \frac{1}{i} = \Theta(\lg x) \qquad \sum_{i=1}^{x} 2^i = \Theta(2^x) \qquad \sum_{i=1}^{x} \frac{1}{2^i} = \Theta(1)$$

- *For functions:*
  - Analyze other functions separately
  - Recursive functions: set up a recurrence and solve
- Overall complexity: largest loop or function call complexity

Back

Analysis exercise

- Find the worst-case complexity of the algorithm below:

```
Input: list: linked list with n nodes
Input: n: length of list
node = list.head
while node ≠ nullptr do
    min = node
    test = node.next
    while test ≠ nullptr do
        if test.value < min.value then
            min = test
        end
        test = test.next
    end
    Swap node.value with min.value
    node = node.next
end
```

Back

Summary: using recursion trees

1. Construct a recurrence
2. Start with n
3. Split n into the recursive cases
4. Repeat down to the base cases
5. Add up complexity on each level
6. Add up complexity for every level

$$T(n) = \underbrace{T(n/2) + T(n/4)}_{\#\text{ and size of recursive calls}} + \underbrace{\Theta(n^2)}_{\text{Complexity of non-recursive code}}$$

$\Theta(n^2)$

$+$

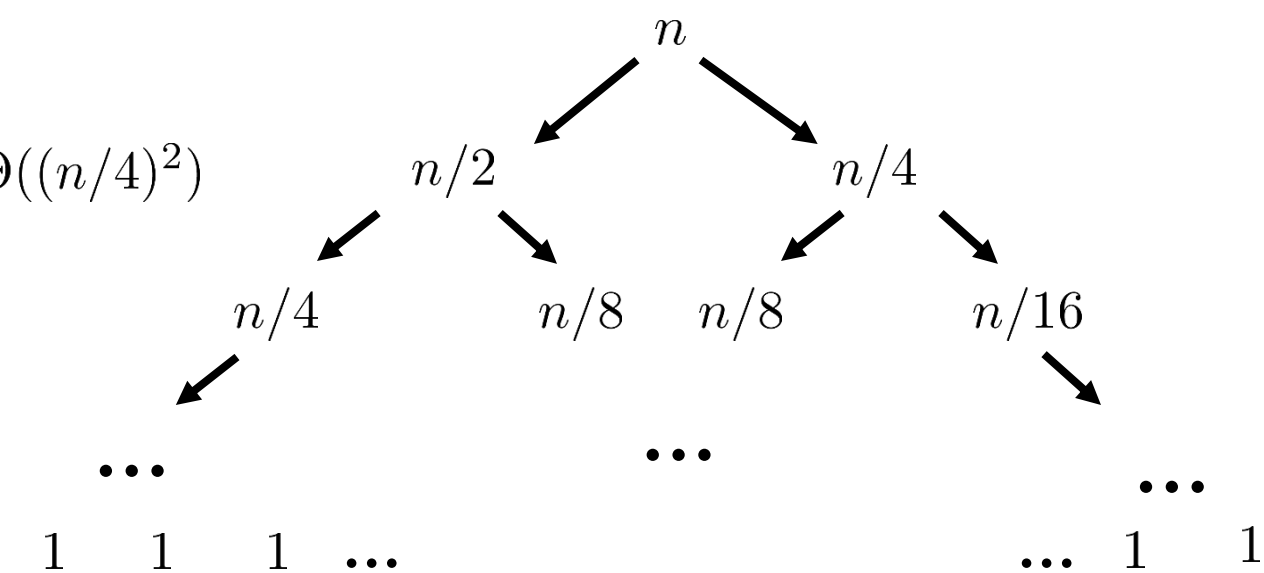
$\Theta((n/2)^2) + \Theta((n/4)^2)$

$+$

$\ldots$

$+$

$x\Theta(1)$



Back

Recursive analysis exercise

1. Give a recurrence that describes the worst-case time complexity for the algorithm below:

Input: A: array of n objects
Input: B: array of n objects
Input: n: number of elements in A and B
Output: minimum distance between two points in A and B according to function $dist$

```
Algorithm: MinDist
if n = 1 then
    return dist(A[1], B[1])
else
    mid = ⌊n/2⌋
    A1 = A[1..mid]
    A2 = A[mid + 1..n]
    B1 = B[1..mid]
    B2 = B[mid + 1..n]
    d11 = MinDist(A1, B1)
    d12 = MinDist(A1, B2)
    d21 = MinDist(A2, B1)
    d22 = MinDist(A2, B2)
    return min{d11, d12, d21, d22}
end
```

Assume $dist$ takes $\Theta(1)$ time

Back

Recursive analysis exercise

2. Sketch a recursion tree for your chosen recurrence

3. Analyze the complexity of this recursion tree

   Hint: $4^{\lg n} = n^2$

Back

Master Theorem

- Powerful theorem for proving complexity of divide-and-conquer algorithms

Master Theorem. If $T(n) = aT(n/b) + f(n)$,

$$T(n) = \begin{cases} \Theta(n^c), & \text{if } f(n) = O(n^{c-\epsilon}) \text{ for some } \epsilon > 0 \\ \Theta(f(n) \lg n), & \text{if } f(n) = \Theta(n^c) \\ \Theta(f(n)), & \text{if } f(n) = \Omega(n^{c+\epsilon}) \text{ for some } \epsilon > 0 \\ & \underline{\text{and}}\ af(n/b) < f(n) \text{ for large } n \end{cases}$$

<u>Four steps to solve:</u>

1. Identify a, b, and $f(n)$
2. Calculate $c = \log_b(a)$
3. Decide case: $f(n)$ vs. n^c: $O(n^{c-\varepsilon})$, $\Theta(n^c)$, $\Omega(n^{c+\varepsilon})$
4. Apply Master Theorem (test regularity if case 3)

Back

Master Theorem exercises

- The table below describes information about five recurrences of the form

$$T(n) = aT(n/b) + f(n)$$

- Please fill in the missing values in the table.

#	a	b	$f(n)$	$T(n)$
1.	3	3	$\Theta(n)$	
2.	8	2	$\Theta(n^2 \lg n)$	
3.		3	$\Theta(n^2)$	$O(n^2)$
4.	4		$\Theta(1)$	$O(\sqrt{n})$
5.	9	3		$\Omega(n^2)$

Back

Stacks and queues

- **Stacks**
  - Support *push()* and *pop()* operations
  - Last-In, First-Out (LIFO) order

- **Queues**
  - Support *enqueue()* and *dequeue()* operations
  - First-In, First-Out (FIFO) order

- **Deques** ("decks")
  - Support all 4 operations

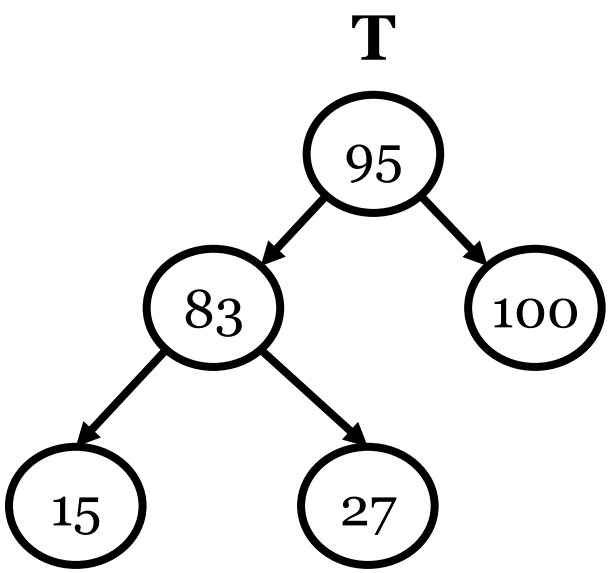
- All three implemented using dynamic arrays
- All operations $\Theta(1)$
  - *enqueue()* and *push()* $\Theta(1)$ *amortized* time

Back

Stack/queue exercise

- Consider the following algorithm for iterating through the elements of a Binary Search Tree:

```
  Input: tree: a BST
1 Algorithm: Iterate
2 nodes = {}
3 Add tree.root to nodes
4 while nodes ≠ ∅ do
5     Print all the elements of nodes
6     t = next element of nodes
7     Add t.left to nodes, unless it's NIL
8     Add t.right to nodes, unless it's NIL
9 end
```



- Assume that Line 5 prints the node values in the order they would be popped or dequeued

1. What is printed by Iterate(T) if nodes is a stack?
2. What is printed by Iterate(T) if nodes is a queue?

Back

Hash table

- Sparse array-based structure
- **Collisions:** more than one element to insert with same hash value
- **Separate chaining**
  - Array of singly-linked lists
  - Prepend when inserting
  - Linked list deletion
  - O(1) worst case for insert
  - Easier to get good performance
- **Open addressing**
  - Insert into "next" available space
  - Mark "absent" when deleting
  - More space efficient
- Both $\Theta(1)$ expected case complexity for all operations
  - Assuming few collisions

Back

Sets

- 3 main operations: Insert(x), Delete(x), Search(x)
- 4/5 secondary operations: Max(), Min(), Successor(x), Predecessor(x)

- Two main implementations with various pros/cons
  - Balanced binary search tree
  - Hash table (expected case)
  - Time complexity, time coefficient (e.g., caching), space (e.g., links vs. no links, empty cells)

- Hash tables: separate chaining vs. open addressing
  - *Load factor:* size of table relative to # of elements

Back

Set complexity

Operation	BBST	Hash table
Search(x)	$O(\lg n)$	$O(1)^{\dagger}$
Delete(x)	$O(\lg n)$	$O(1)^{\dagger}$
Insert(x)	$O(\lg n)$	$O(1)^{*\dagger}$
Min()	$O(\lg n)$	$O(n)^{\dagger}$
Max()	$O(\lg n)$	$O(n)^{\dagger}$
Pred(x)	$O(\lg n)$	$O(n)^{\dagger}$
Succ(x)	$O(\lg n)$	$O(n)^{\dagger}$

* Amortized time
† Expected case

Back

Set exercise

1. Create a table with the complexity of the algorithm below using 4 different set implementations:
   - Sorted and unsorted array, balanced BST, hash table (expected)
   - Assume A, B, and C are *unsorted arrays*

```
Input: A, B: arrays of integers
Input: m, n: length of A and B, respectively
Output: intersection of A and B
Algorithm: Intersect
S = Set()
C = {}
for i = 1 to n do
    S.Insert(B[i])
end
for i = 1 to m do
    if S.Search(A[i]) ≠ NIL then
        Add A[i] to C
    end
end
return C
```

2. Would the runtime change if we swapped A and B?

Back

Maps

- Associates some set off values to another
  - Very useful as look-up table for computations
- Two implementations
- Array-based map
  - *f(i)* stored at *arr[i]*
  - All operations other than initialization take $\Theta(1)$ time
  - Space determined by domain size
- Set-based map (hash map)
  - Set of ordered pairs
  - Complexity determined by set implementation
    - BBST or hash table
  - Space depends on number of values stored
  - Not as efficient if most values in domain appear in map

Back

Priority queues and heaps

- **Priority Queue**
  - Abstract data structure that supports extracting max/min element
  - Main operations (max): Max(), DeleteMax(), Insert(x)
- **Heap:** primary implementation for Priority Queue
  - Array-based complete BST
  - *Heap property*: all children are smaller (larger) than their parent
  - Parent of i is at $i/2$, children are at $2i$ and $2i+1$
  - Helper operations: PercolateUp(i), PercolateDown(i)
    - Shift a value up or down in the tree to satisfy heap property
    - Both: $O(\lg n)$

Operation	Heap
Insert(x)	$O(\lg n)$
Max()	$O(1)$
DeleteMax()	$O(\lg n)$
Heapify	$O(n)$

- No Fibonacci heaps on exam!

Back

Heap exercise

- The algorithm below takes in an index for an element in a heap and reduces its value, then rebalances the heap:

```
Input: heap: heap to modify (min heap)
Input: i: index of value in heap to change
Input: v: new value for heap[i]
Algorithm: heap.ReduceKey(i, v)
heap[i] = v
p = ⌊i/2⌋
while i > 1 and heap[i] < heap[p] do
    Swap heap[i] and heap[p]
    i = p
    p = ⌊i/2⌋
end
```

1. What is the result of calling ReduceKey(9, 17) on the min heap [14, 21, 22, 26, 38, 54, 42, 29, 75, 89]?
2. What is the worst-case time complexity of ReduceKey(i, j) on a heap with n values? Justify your answer.

Back

Union-Find operations

- **Initialize(n)**
  - Assigns every element to its own partition
  - $O(n)$

```
Algorithm: UnionFind(n)
unionfind = Array(n)
for i = 1 to n do
    unionfind[i] = i
end
Let size be an array of n ones
return unionfind
```

- **Find(x)**
  - Follow links to partition ID (root)
  - Recursively point to root
  - $O(\alpha(n))$
  - Generally less than 5 for conceivable n

```
Algorithm: Find(x)
if unionfind[x] ≠ x then
    id = Find(unionfind[x])
    unionfind[x] = id
end
return unionfind[x]
```

- **Union(a, b)**
  - Find root of both sides
  - Point smaller tree to larger
  - $O(\alpha(n))$

```
Algorithm: Union(a, b)
ra = Find(a)
rb = Find(b)
if size[ra] > size[rb] then
    Swap ra and rb
end
unionfind[ra] = rb
size[rb] = size[rb] + size[ra]
```

Back

Union-Find exercise

- **Problem:** blob counting
- **Input:** an n by n matrix of integers 1-4
- **Output:** number of contiguous regions of the same integer
  - Contiguous: cells adjacent horizontally or vertically
- **Example:** $n = 5$, 4 blobs

1	1	3	3	3
1	2	1	3	3
2	2	1	1	3
2	2	1	3	3
2	1	1	1	3

1. Design an algorithm to count blobs
2. Analyze its complexity

Back

Algorithm design example

- What data structure is useful for the following solution to the *knapsack problem*?

- **Problem:** knapsack problem (a.k.a., subset sum)
  - **Input:** various chemicals with *weight* and *value*, and weight limit w
  - **Output:** amount of each chemical to take with weight at most w and maximum value (fractional values allowed)

- **Algorithm strategy**
  - Add the most expensive chemical
  - Repeat until weight exceeds w
  - Take as much of the last chemical to hit the weight limit

Back

Sorting

- Three quadratic algorithms
  - SelectionSort, BubbleSort, InsertionSort
- Three $n \lg n$ algorithms
  - HeapSort, MergeSort, QuickSort
  - Average case for QuickSort: $\Theta(n \lg n)$
- Three non-comparison-based algorithms
  - CountingSort, BucketSort, RadixSort
  - All can sort in $\Theta(n)$ in some cases
- Be familiar with:
  - Pseudocode, complexity (best, worst, average case), advantages and disadvantages
- Lower bound for comparison-based sorting
  - $\Omega(n \lg n)$
  - Proof based on counting execution paths

Back

Selection

- Two selection algorithms
  - QuickSelect, RSelect
- **QuickSelect**
  - Choose random pivot
  - Partition as in QuickSort
  - Recurse left or right
  - *Average complexity:* $O(n)$
  - *Worst-case complexity:* $O(n^2)$
- **RSelect**
  - Split array into sets of 5
  - Calculate medians of each set
  - Recursively find median of medians
  - Partition and recurse with median-of-medians as pivot
  - **Worst-case complexity:** $T(n) = T(n/5) + T(7n/10) + \Theta(n)$
    - $T(n) = \Theta(n)$

Back

Sorting exercise

1. Analyze the worst-case complexity of the algorithm below.
   - Binary search on an array of size x takes $\Theta(\lg x)$ time
2. How does it compare to InsertionSort?

Input: $data$: array of integers
Input: n: size of $data$
Output: permutation of $data$ such that
$data[1] \leq data[2] \leq \ldots \leq data[n]$

```
Algorithm: BinaryInsertionSort
for i = 2 to n do
    ins = data[i]
    Use binary search to find where to insert ins into
      data[1..i − 1]
    Insert ins into data[1..i − 1] at that location
end
return data
```

Coming up

- **Midterm Exam** will be next week
  - Single page of notes
    - Double-sided, standard paper size
  - Submit **Feedback Form** for bonus homework points
  - **Practice exam** to be posted
  - *Optional review session* Monday at 4pm via Teams

- *Post-exam:* algorithm design